

Plume Network: PR 101

Security Review

Cantina Solo review by:
Jiri123, Associate Security Researcher

July 3, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Composer <code>_send</code> reverts on every asset send when <code>ASSET_OFT</code> is the CCTP relayer	4
3.2	Low Risk	4
3.2.1	Keep-collateral full repay reverts on interest accrual	4
3.2.2	Re-enabling performance fees during a drawdown leaves the hurdle clock stale and inflates the next threshold	4
3.2.3	Instant-redeem path skips the zero-receiver guard, burning redeemed assets to <code>0xdead</code>	5
3.3	Informational	5
3.3.1	<code>MIN_HOOK_DATA_LENGTH</code> undercounts the rebuilt hook by one field (440 instead of 472)	5

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Plume is a public, EVM-compatible blockchain built for the next evolution of Real World Assets (RWAs).

From Jun 25th to Jun 28th the security researchers conducted a review of [nest-contracts](#) on commit hash 52c05059. A total of **5** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	3	3	0
Gas Optimizations	0	0	0
Informational	1	1	0
Total	5	5	0

2.1 Scope

The security review had the following components in scope for [nest-contracts](#) on commit hash 52c05059:

```
contracts
├── accountant
│   ├── NestAccountant.sol
│   ├── NestHubAccountant.sol
│   └── NestSpokeAccountant.sol
├── cctp
│   └── NestCCTPRelayer.sol
├── libraries
│   └── nest-vault
│       └── NestVaultCoreTypes.sol
├── morpho
│   ├── libraries
│   │   ├── BundleBuildLib.sol
│   │   └── BundleCalldataLib.sol
│   └── types
│       └── Errors.sol
├── ovault
│   └── NestVaultComposer.sol
├── pendle
│   └── BoringVaultSY.sol
├── types
│   └── Errors.sol
└── upgradeable
    ├── ovault
    │   └── VaultComposerAsyncUpgradeable.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Composer `_send` reverts on every asset send when `ASSET_OFT` is the CCTP relayer

Severity: Medium Risk

Context: [VaultComposerAsyncUpgradeable.sol#L528](#)

Description: `_send` reads `IAppCore(_oft).peers(...)` unconditionally. For asset sends `_oft` is the CCTP relayer (the `ASSET_OFT`), which implements `IOfT` but not `IAppCore`, so the call reverts before reaching `send()`. Every composer asset send (`_finishRedeemAndSend` and instant `_redeemAndSend`) then reverts whenever the relayer is the `ASSET_OFT`, blocking CCTP redemption delivery.

Recommendation: Add `peers(uint32)` to `NestCCTPRelayer` (e.g. derived from CCTP's `remoteTokenMessengers`, encoding the per-chain address width). Add a regression test that wires the real relayer as `ASSET_OFT` without mocking `peers()`.

Plume Network: Fixed in [PR 108](#).

Jiri: Fix verified.

3.2 Low Risk

3.2.1 Keep-collateral full repay reverts on interest accrual

Severity: Low Risk

Context: [BundleCalldataLib.sol#L578-L580](#)

Description: A keep-collateral full repay (`target.loan==0`, `withdrawCollateral==0`) is encoded with `repayShares=type(uint256).max`. Because no collateral is withdrawn, the bundle uses no flash loan and no redeem, so the owner funds exactly `ma.repay`, which is the un-buffered build-time debt `D`. The +10bps buffer is intentionally skipped on this path, so the adapter ends up holding only `D`.

At execution, `shares=max` tells Morpho to repay the position's current debt `D'`, which is `D` plus any interest accrued since the bundle was built. Morpho pulls `D'` from the adapter. If interest accrued (`D'>D`), the adapter is short and the repay reverts.

Collateral-withdrawing full exits use the same `shares=max`, but they receive the +10bps buffer at [BundleBuildLib.sol:115](#) precisely to cover this accrual. The keep-collateral path has no such headroom, so it is safe only when the bundle is built and executed in the same block (`D'==D`). An off-chain build with delayed execution reverts.

Recommendation: Document that keep-collateral full repayments must be built and executed in the same block, since they are not safe under delayed execution. Alternatively, let the owner opt into a small interest buffer on this path (fund `ma.repay.applyBuffer()` from their own balance and sweep the surplus back) so the close survives accrual without forcing a flash loan.

Plume Network: Fixed in [PR 108](#).

Jiri: Fix verified.

3.2.2 Re-enabling performance fees during a drawdown leaves the hurdle clock stale and inflates the next threshold

Severity: Low Risk

Context: [NestHubAccountant.sol#L621-L624](#)

Description: When the performance fee is re-enabled (set from zero back to a non-zero value), `updatePerformanceFee` only resets `hwmLastUpdateTimestamp` inside the `lastGrossRate>highWaterMark` ratchet branch. If this happens during a drawdown (`lastGrossRate<=highWaterMark`), that branch is skipped and the timestamp stays stale. The next `updateExchangeRate` then accrues the hurdle over the whole disabled period, inflating the performance-fee threshold so the manager under-collects on the recovery.

Recommendation: Reset `hwmLastUpdateTimestamp` unconditionally whenever the performance fee is re-enabled, keeping the HWM ratchet-up-only.

Plume Network: Fixed in [PR 108](#).

Jiri: Fix verified.

3.2.3 Instant-redeem path skips the zero-receiver guard, burning redeemed assets to `0xdead`

Severity: Low Risk

Context: [VaultComposerAsyncUpgradeable.sol#L326-L328](#)

Description: The `to==bytes32(0)` guard sits after the `InstantRedeem` early-return, so it only protects the async request/finish/update paths. An instant redeem (and the direct `redeemAndSend` / `depositAndSend` entrypoints) reaches `_send` with an unvalidated `to`. `_fitsAddressType(0)` is true, so a zero receiver passes the width check, and on a remote destination the redeemed assets are sent to `address(0)` / `0xdead` with no source-side refund.

Recommendation: Move the `to==bytes32(0)` check above the `InstantRedeem` branch, ideally to the top of `_send` (and the base sync `_send`) so every path, including the direct entrypoints, fails closed on a zero receiver.

Plume Network: Fixed in [PR 108](#).

Jiri: Fix verified.

3.3 Informational

3.3.1 `MIN_HOOK_DATA_LENGTH` undercounts the rebuilt hook by one field (440 instead of 472)

Severity: Informational

Context: [NestCCTPRelayer.sol#L65-L67](#)

Description: The constant is derived from a 3-field args tuple, but `_validateHookData` runs on the rebuilt hook, whose args tuple has 4 fields (`bytes32depositor,uint256,SendParam,address`). The real minimum is 472 (tuple head 128, not 96), so the 440 guard accepts buffers that are too short for a valid rebuilt hook. There is no security impact, since `_decodeNestHookData` already fails closed on genuinely short buffers; the constant and its comment are simply wrong.

Recommendation: Set `MIN_HOOK_DATA_LENGTH` to 472 and fix the derivation comment to count the 4-field tuple (head 128).

Plume Network: Fixed in [PR 108](#).

Jiri: Fix verified.